Сетевая модель и контракты взаимодействия

Table of contents

Сетевая модель и контракты взаимодействия . . . 1
- Протокол — это договор . . . 1
- Модель OSI: семь уровней договорённостей . . . 2
  - Как уровни строятся друг на друге . . . 2
  - Практическое упрощение: TCP/IP . . . 3
  - TCP vs UDP: два подхода к доставке . . . 3
- HTTP — прикладной протокол веба . . . 4
  - Структура HTTP-запроса . . . 4
  - Структура HTTP-ответа . . . 4
  - HTTP-методы . . . 4
  - Коды статуса . . . 5
  - От HTTP к REST . . . 5
- Контракт API: от договорённости к спецификации . . . 6
  - OpenAPI (Swagger) — спецификация REST API . . . 6
  - Protocol Buffers — спецификация gRPC . . . 7
  - GraphQL SDL — спецификация GraphQL API . . . 8
  - Общий принцип: спецификация = единый источник правды . . . 9
- Связь уровней OSI и контрактов . . . 9

Сетевая модель и контракты взаимодействия

Прежде чем выбирать конкретный прикладной протокол (REST, gRPC, GraphQL), необходимо понять **как вообще устроена передача данных между двумя программами** и **что такое контракт взаимодействия**.

Протокол — это договор

Протокол — это набор правил, по которым две стороны обмениваются данными. Без договорённости о формате, порядке и значении сообщений коммуникация невозможна.

Пример из университетской жизни: чтобы получить справку в деканате, студент должен следовать протоколу — заполнить заявление по установленной форме (формат сообщения), подать его в определённое окно в рабочие часы (канал и адресация), дождаться обработки и получить ответ с печатью (формат ответа и подтверждение). Если студент напишет заявление в свободной форме, придёт не в тот кабинет или забудет номер студенческого — протокол нарушен и результата не будет. Между компьютерами происходит то же самое, только договорённости формализованы и многоуровневы.

Ключевое свойство: каждый уровень коммуникации опирается на свой протокол, и верхние уровни **не обязаны знать**, как именно работают нижние. Это называется **инкапсуляцией уровней**.

Модель OSI: семь уровней договорённостей

Модель OSI (Open Systems Interconnection) описывает сетевое взаимодействие как стек из семи уровней. Каждый уровень решает свою задачу и предоставляет сервис уровню выше.

Как уровни строятся друг на друге

Каждый следующий уровень — это новый договор, который опирается на возможности предыдущего. Построим стек снизу вверх:

1. **Есть провод** (L1 — физический). Два компьютера соединены медным кабелем. Договариваемся: напряжение выше +0.5 В — это «1», ниже −0.5 В — это «0»; передаём с частотой 100 МГц.
   Вход: электрический сигнал `+V −V −V +V +V −V −V +V`. **Выход:** поток битов `10011001`.

2. **Разбиваем биты на кадры** (L2 — канальный). Непрерывный поток битов бесполезен — неясно, где начало сообщения и кому оно адресовано. Договариваемся разбивать поток на **кадры** (frames): в начале каждого кадра — преамбула (фиксированная последовательность битов для синхронизации), затем MAC-адрес получателя (`AA:BB:CC:DD:EE:FF`), MAC-адрес отправителя, длина, полезная нагрузка и контрольная сумма. Сетевая карта читает MAC-адрес — если это её адрес, принимает кадр, иначе игнорирует.
   Вход: поток битов `10011001...`. **Выход:** кадр `[преамбула][dst: AA:BB:CC:DD:EE:FF][src: 11:22:33:44:55:66][данные][CRC]`, доставленный конкретному устройству в локальной сети.

3. **Добавляем маршрутизацию между сетями** (L3 — сетевой). MAC-адреса работают только в пределах одного сегмента сети. Чтобы связать сети, оборачиваем данные в **IP-пакет**: заголовок содержит IP-адрес отправителя (`192.168.1.10`) и получателя (`93.184.216.34`). Маршрутизатор на границе сети читает IP-адрес назначения, находит следующий узел в таблице маршрутизации и пересылает пакет дальше — и так по цепочке до целевой сети.
   Вход: кадр с данными. **Выход:** IP-пакет `[src: 192.168.1.10][dst: 93.184.216.34][данные]`, доставленный на нужный компьютер через произвольное количество промежуточных сетей.

4. **Доставляем данные конкретной программе** (L4 — транспортный). На одном компьютере работают десятки процессов: веб-сервер, база данных, почтовый клиент. IP доставил пакет на машину, но кому именно? Договариваемся о **портах** — числовых идентификаторах (0–65535). TCP добавляет к каждому сегменту порт отправителя и получателя (например, `:443`), а также порядковый номер, подтверждение доставки и контрольную сумму. Если сегмент потерялся — TCP замечает пропуск в номерах и запрашивает повторную отправку.

Вход: IP-пакет, доставленный на машину. **Выход:** TCP-сегмент `[src port: 51234][dst port: 443][seq: 1][ack: 1][данные]`, надёжно доставленный конкретному процессу.

5. **Шифруем и форматируем** (L5/L6 — сеансовый и представления). Данные доставляются, но в открытом виде — любой промежуточный узел может их прочитать. Договариваемся о TLS: клиент и сервер выполняют **handshake** (обмениваются сертификатами и ключами), после чего весь поток шифруется. Также на этом уровне фиксируется кодировка (UTF-8) и формат сериализации (JSON, XML, бинарный).
   Вход: поток байтов `GET /users/42...` в открытом виде. **Выход:** зашифрованный поток `17 03 03 00 1A 8b 4e ...` — без ключа содержимое нечитаемо.

6. **Определяем смысл сообщений** (L7 — прикладной). Все нижние уровни обеспечивают доставку, но не определяют **что означает** переданное сообщение. На прикладном уровне фиксируем протокол — например, HTTP: клиент отправляет `GET /users/42 HTTP/1.1`, сервер отвечает `200 OK` с телом `{"id": 42, "name": "Alice"}`. Метод, путь, заголовки и код статуса — всё это договорённости прикладного протокола.

Каждый уровень оборачивает данные в свой «конверт» (заголовок) при отправке — это **инкапсуляция**. На принимающей стороне каждый уровень снимает свой заголовок и передаёт содержимое выше — это **декапсуляция**.

Практическое упрощение: TCP/IP

На практике чаще используют четырёхуровневую модель TCP/IP, которая объединяет уровни OSI:

Уровень TCP/IP	Задача	Соответствие OSI
Прикладной	Протокол приложения (HTTP, DNS, ...)	L5–L7
Транспортный	Доставка данных процессу (TCP/UDP)	L4
Сетевой	Маршрутизация между сетями (IP)	L3
Канальный	Доставка в локальной сети	L1–L2

Для разработчика ключевые уровни — транспортный и прикладной. Именно здесь принимаются архитектурные решения: выбор между TCP и UDP, между HTTP/1.1 и HTTP/2, между текстовым и бинарным форматом сообщений.

TCP vs UDP: два подхода к доставке

TCP (Transmission Control Protocol) — надёжный, упорядоченный, с контролем потока. Гарантирует доставку и порядок. Используется для HTTP, gRPC, баз данных — везде, где потеря данных недопустима.

UDP (User Datagram Protocol) — минимальный, без гарантий доставки и порядка. Используется для видеозвонков, онлайн-игр, DNS — там, где скорость важнее полноты.

Выбор транспорта определяется требованиями прикладного протокола: HTTP **требует** TCP, а потоковое видео **допускает** потери и выбирает UDP.

HTTP — прикладной протокол веба

TCP умеет надёжно доставлять поток байтов между двумя процессами. Но он ничего не знает о **смысле** этих байтов — для TCP строка `GET /users/42` и случайный набор символов одинаковы. Нужен следующий договор: как именно клиент формулирует запрос и как сервер отвечает.

Этот договор — **HTTP** (HyperText Transfer Protocol). Он фиксирует текстовый формат сообщений поверх TCP-соединения.

Структура HTTP-запроса

Клиент отправляет текстовое сообщение строго определённой структуры:

```
GET /users/42 HTTP/1.1
Host: api.example.com
Accept: application/json
Authorization: Bearer eyJhbGciOi...
```

Компоненты запроса: - **Стартовая строка:** метод (`GET`), путь (`/users/42`), версия протокола (`HTTP/1.1`) - **Заголовки:** пары ключ-значение с метаинформацией — куда обращаемся (`Host`), какой формат ответа ожидаем (`Accept`), как авторизуемся (`Authorization`) - **Пустая строка:** разделитель между заголовками и телом - **Тело** (опционально): полезная нагрузка — например, JSON при создании ресурса

Структура HTTP-ответа

Сервер отвечает в аналогичном формате:

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 42

{"id": 42, "name": "Alice", "email": "alice@example.com"}
```

Компоненты ответа: - **Статусная строка:** версия протокола, код статуса (`200`), текстовое пояснение (`OK`) - **Заголовки:** тип содержимого (`Content-Type`), размер тела (`Content-Length`) - **Тело:** полезная нагрузка ответа

HTTP-методы

HTTP определяет **методы** — глаголы, описывающие намерение клиента:

Метод	Семантика	Тело запроса
`GET`	Получить ресурс. Не изменяет состояние сервера.	Нет
`POST`	Создать новый ресурс или выполнить операцию.	Да
`PUT`	Полностью заменить ресурс.	Да

Метод	Семантика	Тело запроса
`PATCH`	Частично обновить ресурс.	Да
`DELETE`	Удалить ресурс.	Обычно нет

Методы — это именно **договорённость о намерении**. Технически через `GET` можно удалять данные, но это нарушает протокол и ломает кеширование, прокси и поведение браузера.

Коды статуса

Код статуса сообщает клиенту **результат** обработки запроса. Коды сгруппированы по сотням:

Диапазон	Значение	Примеры
`1xx`	Информационный — запрос принят, обработка продолжается	`101 Switching Protocols`
`2xx`	Успех — запрос обработан корректно	`200 OK`, `201 Created`, `204 No Content`
`3xx`	Перенаправление — ресурс доступен по другому адресу	`301 Moved Permanently`, `304 Not Modified`
`4xx`	Ошибка клиента — запрос некорректен	`400 Bad Request`, `401 Unauthorized`, `404 Not Found`
`5xx`	Ошибка сервера — запрос корректен, но сервер не смог обработать	`500 Internal Server Error`, `503 Service Unavailable`

От HTTP к REST

HTTP сам по себе — это протокол передачи сообщений. Он не навязывает, как организовать API: можно сделать единственный эндпоинт `POST /api` и передавать всё в теле, а можно использовать десятки путей без какой-либо системы.

REST (Representational State Transfer) — это архитектурный стиль, который задаёт **правила организации API поверх HTTP**:

- **Ресурсы:** каждая сущность (пользователь, заказ, товар) адресуется собственным URI: `/users/42`, `/orders/7`.
- **Методы = операции:** HTTP-методы отражают действие над ресурсом — `GET /users/42` читает, `DELETE /users/42` удаляет.
- **Коды статуса = результат:** `201 Created` при создании, `404 Not Found` если ресурс не существует, `409 Conflict` при конфликте.

- **Stateless:** каждый запрос содержит всю информацию для обработки — сервер не хранит состояние между запросами.

Пример CRUD-операций над ресурсом «пользователи»:

Операция	Запрос	Тело	Ответ
Список	`GET /users`	—	`200 OK` + массив JSON
Чтение	`GET /users/42`	—	`200 OK` + объект JSON
Создание	`POST /users`	`{"name": "Bob"}`	`201 Created` + объект с `id`
Обновление	`PUT /users/42`	`{"name": "Bob2"}`	`200 OK` + обновлённый объект
Удаление	`DELETE /users/42`	—	`204 No Content`

REST — не стандарт и не спецификация, а набор принципов. Конкретный API может следовать им строго или частично. Именно поэтому существует потребность в **формальных спецификациях**, которые фиксируют контракт однозначно.

Контракт API: от договорённости к спецификации

Протокол определяет **как** передаются данные. Но для взаимодействия двух сервисов этого недостаточно — нужен **контракт**: формальное описание того, **какие операции доступны, какие данные они принимают и возвращают**.

Без контракта разработчики двух сервисов вынуждены договариваться устно или читать чужой код. Это не масштабируется: при десятках сервисов и нескольких командах устные договорённости ломаются.

Поэтому существуют **формальные спецификации API** — машиночитаемые описания контрактов.

OpenAPI (Swagger) — спецификация REST API

OpenAPI описывает REST API в формате YAML или JSON: эндпоинты, методы, параметры, тела запросов и ответов, коды ошибок.

```yaml
openapi: "3.0.3"
info:
  title: User Service
  version: "1.0"
paths:
  /users/{id}:
```

```yaml
    get:
      summary: Получить пользователя по ID
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
      responses:
        "200":
          description: Пользователь найден
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/User"
        "404":
          description: Пользователь не найден
components:
  schemas:
    User:
      type: object
      properties:
        id:
          type: integer
        name:
          type: string
        email:
          type: string
          format: email
```

Из OpenAPI-спецификации автоматически генерируются: документация (Swagger UI), клиентские SDK, серверные заглушки (stubs) и валидаторы запросов.

Protocol Buffers — спецификация gRPC

gRPC использует `.proto`-файлы для описания сервисов и сообщений. Формат строго типизирован и компилируется в код на целевом языке.

```
syntax = "proto3";

service UserService {
  rpc GetUser (GetUserRequest) returns (User);
  rpc ListUsers (ListUsersRequest) returns (stream User);
}
```

```
message GetUserRequest {
  int32 id = 1;
}

message ListUsersRequest {
  int32 page_size = 1;
  string page_token = 2;
}

message User {
  int32 id = 1;
  string name = 2;
  string email = 3;
}
```

Из `.proto`-файла генерируются клиент и сервер — разработчику остаётся реализовать бизнес-логику. Контракт гарантирует совместимость: если поле добавлено корректно (новый номер), старые клиенты продолжат работать.

GraphQL SDL — спецификация GraphQL API

GraphQL описывает API через схему (Schema Definition Language), которая определяет типы данных и доступные запросы.

```
type User {
  id: ID!
  name: String!
  email: String!
  posts: [Post!]!
}

type Post {
  id: ID!
  title: String!
  content: String!
}

type Query {
  user(id: ID!): User
  users(limit: Int = 10): [User!]!
}
```

Схема является и документацией, и контрактом, и инструментом валидации: сервер отклонит запрос, который не соответствует схеме.

Общий принцип: спецификация = единый источник правды

Во всех трёх подходах формальная спецификация служит единым источником правды (single source of truth) для всех участников:

Свойство	OpenAPI	Proto (gRPC)	GraphQL SDL
Формат	YAML / JSON	`.proto`	SDL (текст)
Типизация	JSON Schema	Строгая, бинарная	Строгая, граф
Кодогенерация	Клиент, документация	Клиент + сервер	Клиент, валидация
Версионирование	URL (`/v1`, `/v2`)	Номера полей	Эволюция схемы
Транспорт	HTTP/1.1+	HTTP/2	HTTP (обычно)

Связь уровней OSI и контрактов

Спецификации API работают на прикладном уровне (L7), но опираются на весь стек:

- **L4 (TCP)** обеспечивает надёжную доставку — без этого ни один API-вызов не завершится.
- **L6/L5 (TLS)** обеспечивают шифрование — без этого контракт выполняется, но данные видны третьим лицам.
- **L7 (HTTP/HTTP2)** задаёт правила обмена сообщениями — заголовки, методы, коды статуса.
- **Спецификация API** (OpenAPI, proto, SDL) фиксирует **бизнес-семантику** поверх HTTP: какие операции существуют, что принимают и возвращают.

Таким образом, каждый уровень — это свой договор, и прикладной контракт является верхним слоем этой пирамиды договорённостей.